

Lecciones aprendidas

1. **Validación de datos es fundamental:** Una vez se ha creado una base de datos, es necesario de que se pongan limitaciones al llenar los formularios que luego creen entradas en la base de datos, de forma que se tenga que especificar que partes si tienen que incluirse y cuales no se pueden repetir, lo que reduce los errores que esto puede causar. Como ejemplo, podemos decir que se nos olvidó asegurar de que el correo tiene que ser único, esto puede causar problemas debido a que esto se ocupa para ingresar sesión, lo que pudiera causar que haya 2 personas en la base de datos que ocupen el mismo correo, pero con distintos roles.
2. **El control de roles mejora la seguridad y organización de la plataforma:** Tener usuarios con roles específicos facilita el uso de la plataforma y mejora la seguridad de esta. Esto es debido a que, al limitar lo que puede hacer cada usuario, nos aseguramos de que solo ciertos usuarios puedan realizar ciertas acciones. De la misma manera, los roles de los usuarios afectan la información a la cual pueden acceder ellos y, por ende, la seguridad de esta información, ya que no todos tendrán acceso a toda la información disponible. Como ejemplo, en nuestra plataforma, solo el usuario Administrador tiene acceso a la creación de nuevos usuarios, por lo que esta función esencial tiene una mejor seguridad que la de subir un documento, lo cual pueden hacer todos los usuarios.
3. **La planificación reduce los errores y evita tener que rehacer partes:** Si se planea un proyecto de forma correcta, incluyendo partes como el alcance del proyecto, los limites y los requerimientos funcionales y no funcionales, entonces se pueden evitar o reducir errores en el mismo proyecto (como no dejar una parte clara). Aparte de esto, si se planea todo bien, se tiene una buena idea de que es lo que se necesita en el proyecto y se evita que se programe una parte y que luego la tenga que cambiar o eliminar porque no cumple con lo antes especificado. El hecho de planificar todo esto desde el inicio nos ayudó bastante, ya que, a la hora de empezar el diseño y la programación de la plataforma, ya teníamos claro que teníamos que hacer y como todo debería de interactuar.
4. **Las pruebas permiten encontrar errores antes de la entrega:** Siempre van a haber errores en un programa, y es debido a esto, por lo que tenemos que realizar pruebas para asegurarnos de que captemos los errores y los corrijamos antes de que se haga la entrega final. Esto es porque, sale más fácil encontrar y corregir los errores cuando no hay usuarios, que cuando ya se cuentan con esto, debido a que se tuviera que desconectar la plataforma mientras se realiza el mantenimiento de esta. Estas pruebas nos ayudaron a encontrar varios errores que teníamos en nuestro programa, específicamente la validación de datos antes de ingresar

dichos datos a la base de datos (como el de dejar ciertos campos vacíos o que estos se pudieran repetir).

5. **La arquitectura en capas ayuda a organizar el proyecto:** La división de nuestra arquitectura en capas fue de gran utilidad, debido a que pudimos separar toda nuestra aplicación en 3 capas distintas (capa de presentación UI, capa lógica y capa de datos) y enfocarnos en cada una de ellas de forma separada antes de integrar todo. Para nuestro proyecto, la capa de presentación lidiaba con la interfaz del usuario, la capa lógica lidiaba con la lógica de las distintas funcionalidades permitidas a nuestros usuarios, y finalmente, la capa de datos lidiaba con las bases de datos y las conexiones a estas por medio de la plataforma. Pudimos enfocarnos en esas 3 capas específicamente, antes de juntarlas todas a una sola plataforma con todas las funcionalidades de las 3 capas.
6. **Mantener una buena documentación facilita el desarrollo de la aplicación y su mantenimiento:** Encontramos que, al realizar la documentación y entregarla a través del ciclo, pudimos volver a revisar dicha documentación cuando tuviéramos dudas y ocuparla para solventarlas. Por ejemplo, si tuviéramos dudas de cómo debería funcionar una parte específica, podíamos regresar a la primera entrega y revisar los alcances y los requisitos funcionales para tener una idea más clara de cómo programarla y por qué se debe de hacer así.